

CS & IT ENGINEERING



Operating System

Deadlock

Lecture – 03

By– Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Deadlock Prevention

Topic

Deadlock Avoidance

Topics to be Covered



Topic

Deadlock Avoidance

Topic

Banker's Algorithm

Topic

Deadlock Detection



Topic : Banker's Algorithm

1. Allocation:
2. Max:
3. Need:
4. Available:



Topic : Banker's Algorithm

1. Let Work and Finish be vectors of length 'm' and 'n' respectively.
Initialize: $Work = Available$
 $Finish[i] = false$; for $i=1, 2, 3, 4 \dots n$
2. Find an i such that both (a) $Finish[i] = false$
(b) $Need_i \leq Work$
if no such i exists goto step (4)
3. $Work = Work + Allocation[i]$
 $Finish[i] = true$ goto step (2)
4. If $Finish[i] = true$ for all i
then the system is in a safe state

#Q.

Process	Allocation A B C D	Max A B C D	Available A B C D	Need A B C D
P1	0 0 1 2	0 0 1 2	1 5 2 0	0 0 0 0
P2	1 0 0 0	1 7 5 0	^{P1} 1 5 3 2	0 7 5 0
P3	1 3 5 4	2 3 5 6	^{P3} 2 8 8 6	1 0 0 2
P4	0 6 3 2	0 6 5 4		0 0 2 2
P5	0 0 1 4	0 6 5 6		0 6 4 2

Safe



Topic : Banker's Algorithm

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P ₀	^{1 1 2} 0 1 0	7 5 3	3 3 2 ^{2 3 0}	7 4 3 ^{6 4 1}
P ₁	2 0 0	3 2 2		1 2 2
P ₂	3 0 2	9 0 2		6 0 0
P ₃	2 1 1	2 2 2		0 1 1
P ₄	0 0 2	4 3 3		4 3 1

No any process can finish

unsafe $\Rightarrow$ request of P₀ rejected

#Q. What will happen if process P0 requests one additional instance of resource type A and two instances of resource type C?

$$\text{request}_{P_0} = \langle 1, 0, 2 \rangle$$

1. if $\text{request}_i \leq \text{need}_i$ goto step 2
else invalid request
2. if $\text{request}_i \leq \text{available}$ goto step 3
else wait
3. $\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i$
 $\text{Need}_i = \text{Need}_i - \text{Request}_i$
 $\text{Available} = \text{Available} - \text{Request}_i$

4. Run safety algo & check if system is safe means request_i is granted; otherwise request_i will be rejected



Topic : Banker's Algorithm

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2 3 2 2	7 4 3
P ₁	2 0 0	3 2 2	↓ safe	1 2 2
P ₂	3 0 2	9 0 2		6 0 0
P ₃	2 2 1 2 1 1	2 2 2		0 0 1 0 1 1
P ₄	0 0 2	4 3 3		4 3 1

#Q. What will happen if process P3 requests one additional instance of resource type B?

$$\text{Request}_3 = \langle 0, 1, 0 \rangle$$

↓
granted



Topic : Resource Request Algorithm

1. If $\text{Request}_i \leq \text{Need}_i$
Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If $\text{Request}_i \leq \text{Available}$
Goto step (3); otherwise, P_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:
 $\text{Available} = \text{Available} - \text{Request}_i$
 $\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i$
 $\text{Need}_i = \text{Need}_i - \text{Request}_i$



Topic : Banker's Algorithm

Process	Allocation	Max	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	

- #Q. What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C ?
What will happen if process P3 requests one additional instance of resource type B?

$$Req_{P1} = \langle 1, 0, 2 \rangle$$

$$Req_{P3} = \langle 0, 1, 0 \rangle$$

Possibilities :-

1. Req_1 and Req_3 both granted
2. No any request granted (both rejected)
3. only Req_1 is granted
4. only Req_3 is granted
5. Either req_1 or req_3 is granted but not both together

Deadlock avoidance (banker's algo) can not be implemented practically.



because knowledge of max to be known priorly



Topic : Deadlock Detection

1. When all resources have single instance $\Rightarrow$ wait-for-graph
2. When resources have multiple instances
 $\hookrightarrow$ Banker's algo



Topic : Deadlock Detection



When all resources have single instance:

Deadlock detection is done using wait-for-graph



Topic : Wait For Graph

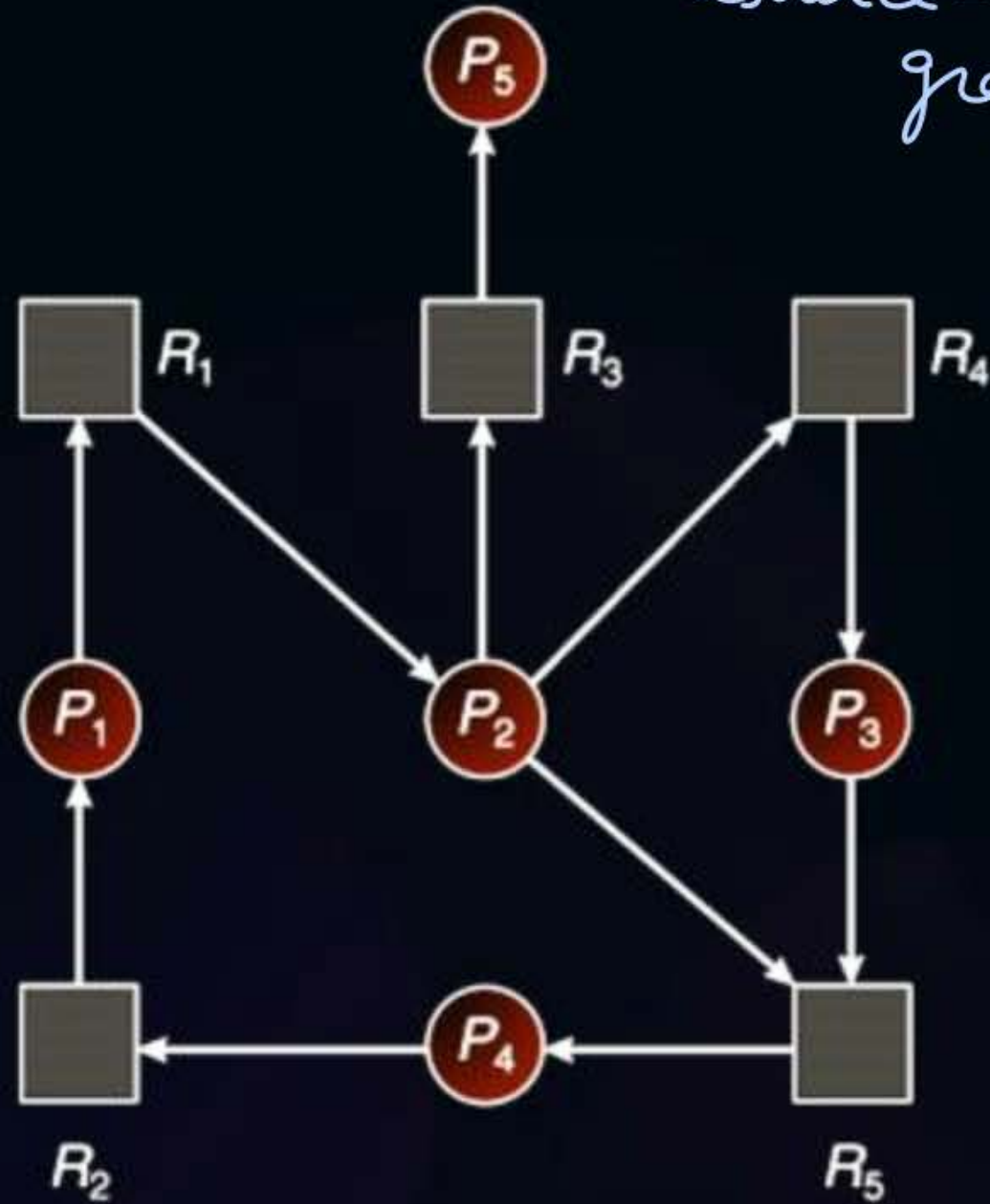


It is created from resource allocation graph



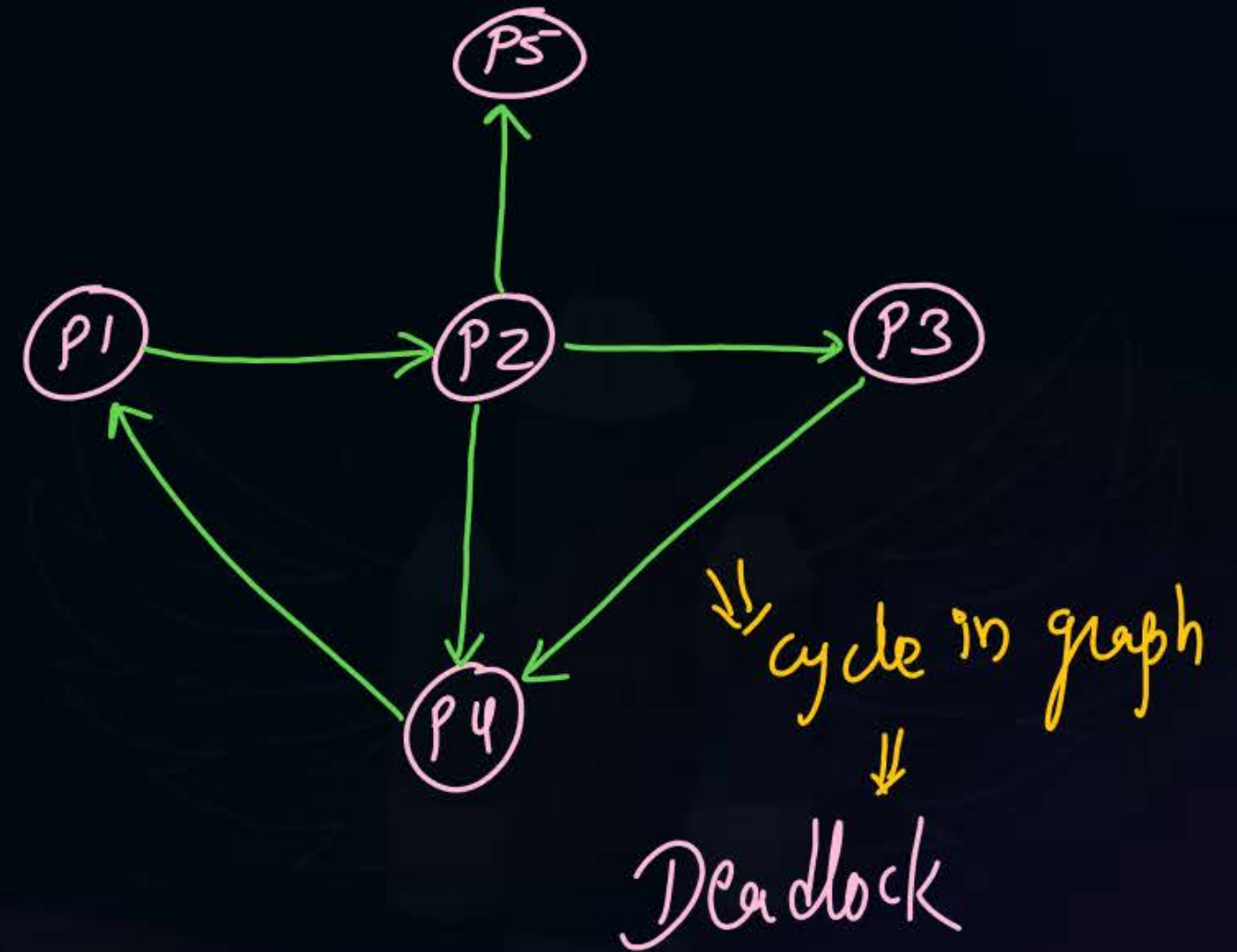
Topic : Wait For Graph

Resource - allocatⁿ
graph

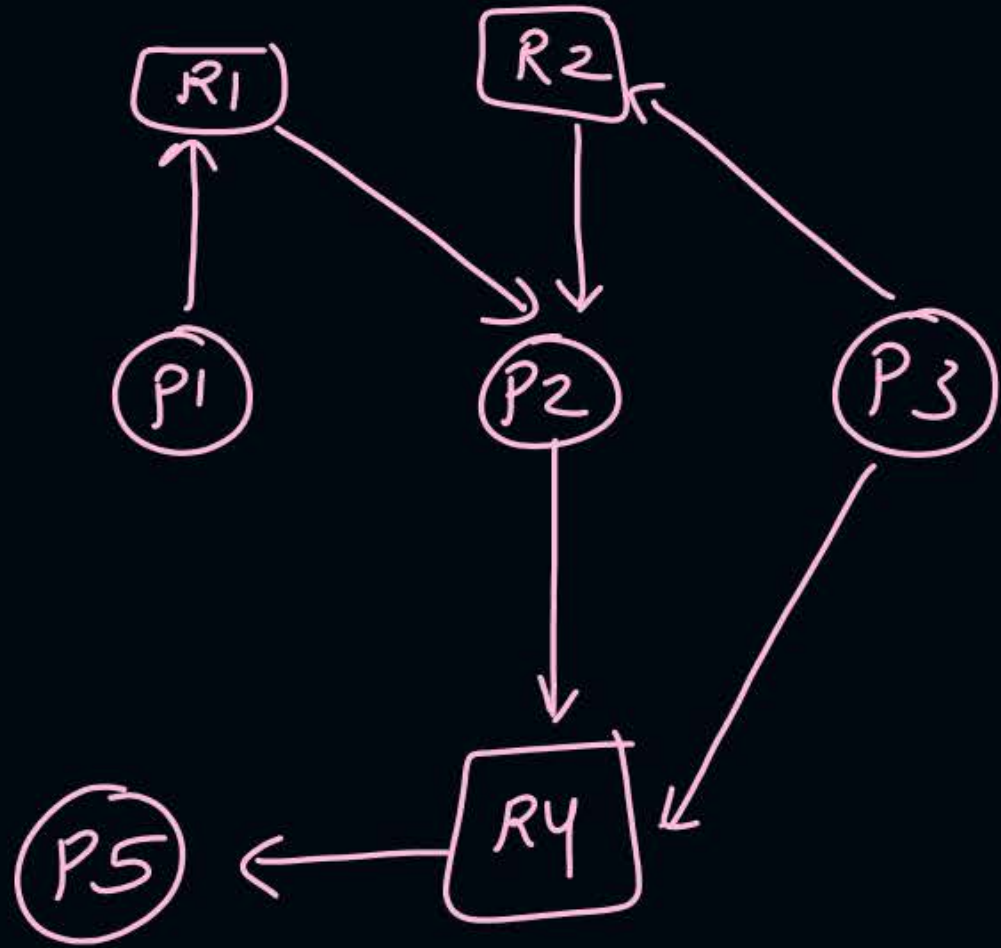


(a)

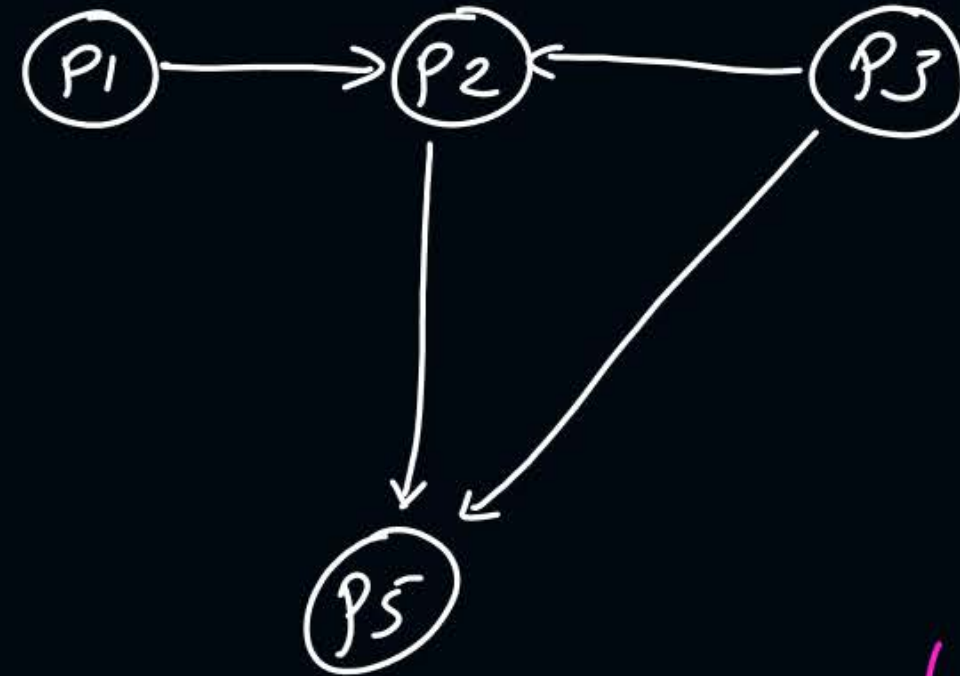
directⁿ of edge
wait - for - graph : -
has only processes



Ques)



wait-for graph:-



no cycle $\Rightarrow$ no deadlock



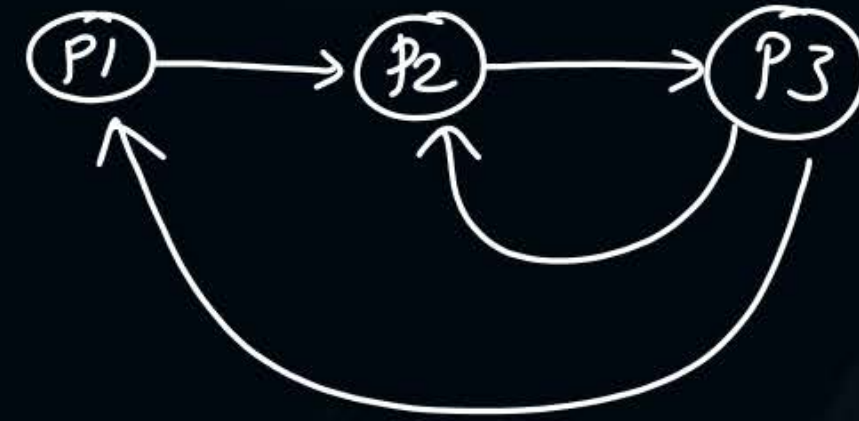
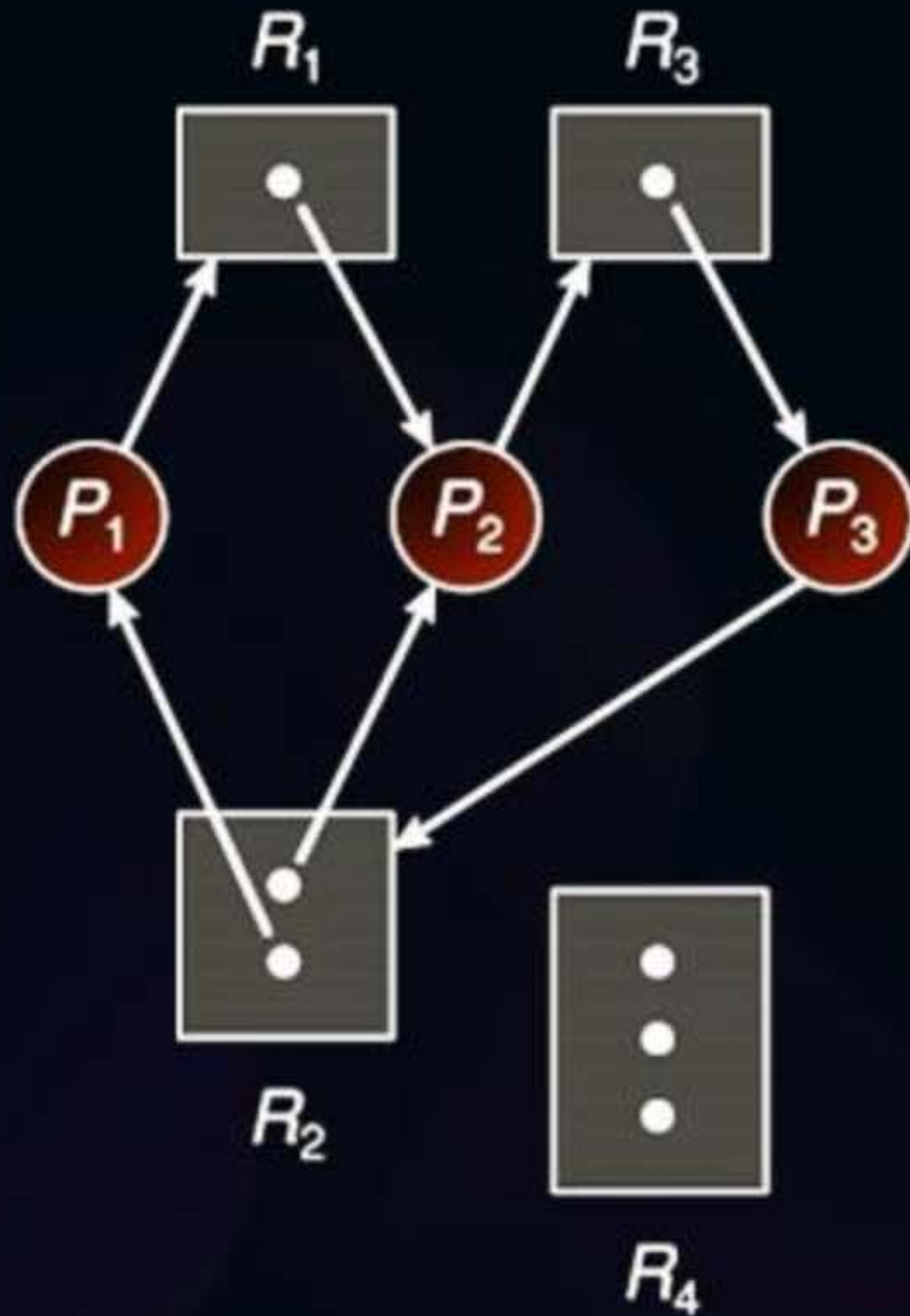
Topic : Wait For Graph



If a resource category contains more than one instance, then the presence of a cycle in the resource-allocation graph indicates the possibility of a deadlock, but does not guarantee one.



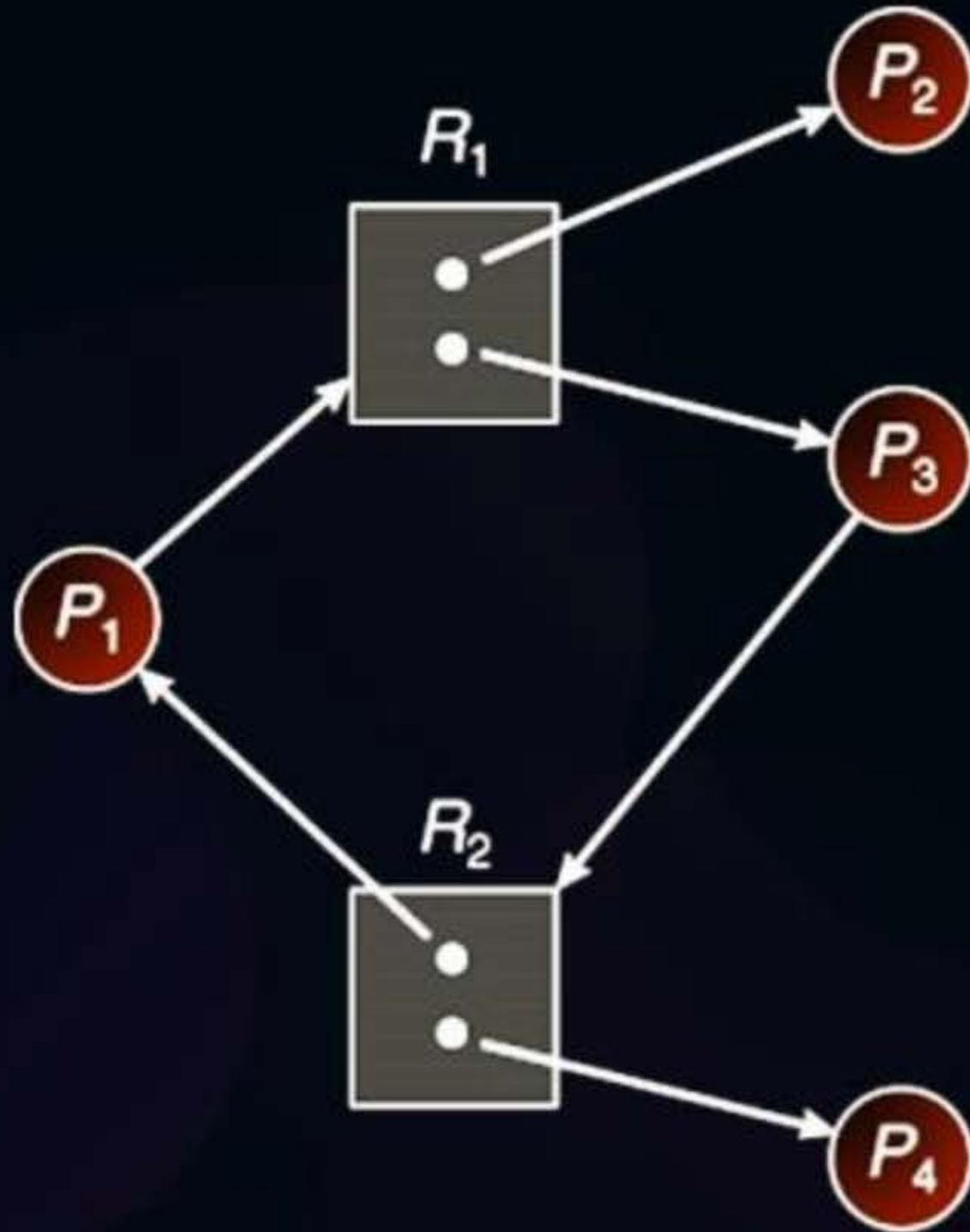
Topic : Wait For Graph: Example



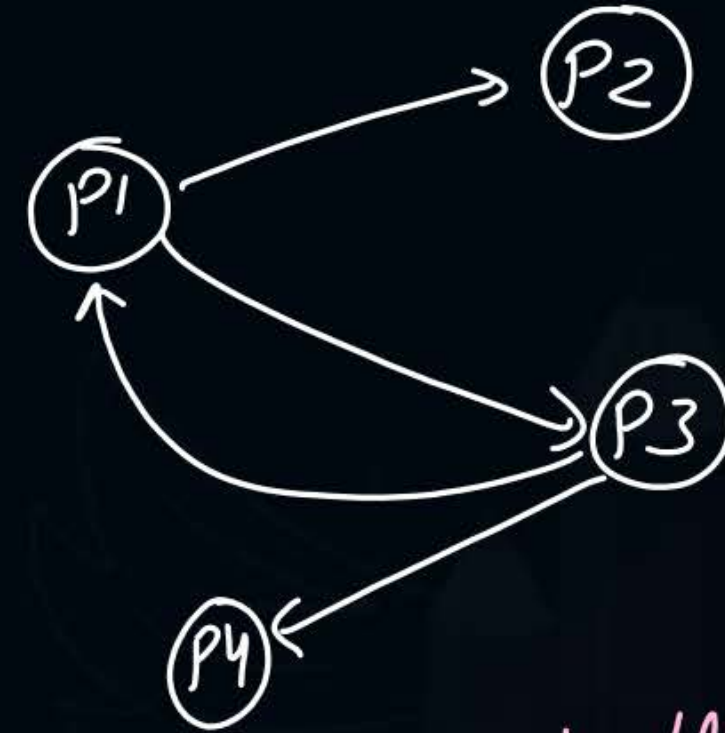
cycle $\Rightarrow$ deadlock



Topic : Wait For Graph: Example



wait-for graph:-



cycle $\Rightarrow$ no deadlock

	Allocation		Request		Available	
	R1	R2	R1	R2	R1	R2
P1	0	1	1	0	0	0
P2	1	0	0	0	After P2	1 0
P3	1	0	0	1	After P4	1 1
P4	0	1	0	0	After P1	1 2
					After P3	2 2
					↓	
					no deadlock	



Topic : Deadlock Detection



When resources have multiple instance:

Deadlock detection is done using a specific algorithm

↓
banker's



Topic : Deadlock Detection Algorithm

	Allocation	Request	Available
	A B C	A B C	A B C
P ₀	0 1 0	0 0 0	0 0 0
P ₁	2 0 0	2 0 2	
P ₂	3 0 3	0 0 0	
P ₃	2 1 1	1 0 0	
P ₄	0 0 2	0 0 2	

← add allocatⁿ of P₀

After P₀ 0 1 0

After P₂ 3 1 3

After P₁ 5 1 3

After P₃ 7 3 4

for which process $Req_i \leq Available$

After P₄ 7 3 6

all processes completed

↓
no deadlock



Topic : Deadlock Detection Algorithm

1. Let Work and Finish be vectors of length m and n respectively.
Initialize $Work = Available$. For $i=0, 1, \dots, n-1$, if $Request_i = 0$, then $Finish[i] = true$; otherwise, $Finish[i] = false$.
2. Find an index i such that both
 - (a) $Finish[i] == false$
 - (b) $Request_i \leq Work$If no such i exists go to step 4.
3. $Work = Work + Allocation_i$
 $Finish[i] = true$
Go to Step 2.
4. If $Finish[i] == false$ for some i , $0 \leq i < n$, then the system is in a deadlocked state.
Moreover, if $Finish[i] == false$ the process P_i is deadlocked.



2 mins Summary

Topic

Deadlock Avoidance

Topic

Banker's Algorithm

Topic

Deadlock Detection



Happy Learning

THANK - YOU